

Dynamic Air Conditioner and Room Simulation

Abdelkader Messlem

Abstract

This report presents a dynamic Simulink model for room temperature and humidity control using an air conditioner. The model contains three main MATLAB Function blocks: an automatic comfort controller, an air conditioner unit, and a room thermal-humidity model. The controller selects fan speed, air direction, and supply air temperature. The air conditioner model calculates airflow, supply air conditions, and electrical power. The room model predicts temperature and humidity using thermal and moisture balance equations.

1 System

Table 1: Main Simulation Blocks

Block	Type	Function
Controller	MATLAB Function	Selects fan speed, air direction, and supply temperature.
Air conditioner	MATLAB Function	Calculates airflow, supply air temperature, supply humidity, and power.
Room model	MATLAB Function	Calculates room temperature and humidity dynamics.
Dynamic states	Integrators	Store airflow, supply temperature, room temperature, wall temperature, furniture temperature, humidity ratio, and energy.

Table 2: Signal Names and Units

Signal	Description	Unit
t	Simulation time	s
T_r	Room temperature	°C
RH_r	Room relative humidity	%
T_w	Wall temperature	°C
T_f	Furniture temperature	°C
w_r	Room humidity ratio	kg/kg
T_{out}	Outdoor temperature	°C
RH_{out}	Outdoor relative humidity	%
$\dot{m}_{ac}$	Air conditioner air mass flow rate	kg/s
T_{sup}	Supply air temperature	°C
RH_{sup}	Supply air relative humidity	%
P_{ac}	Air conditioner electrical power	W
fan_{cmd}	Fan speed command	0 to 3
dir_{cmd}	Air direction command	0 to 1
$T_{sup,cmd}$	Supply temperature command	°C
T_{des}	Desired room temperature calculated inside controller	°C
RH_{des}	Desired room relative humidity calculated inside controller	%

Table 3: Block Connections

Input	Connected from
controller.t	Clock
controller.T_r	Room temperature feedback
controller.RH_r	Room relative humidity feedback
controller.T_out	Outdoor temperature input
controller.RH_out	Outdoor relative humidity input
ac_unit.fan_cmd	controller.fan_cmd
ac_unit.dir_cmd	controller.dir_cmd
ac_unit.T_sup_cmd	controller.T_sup_cmd
ac_unit.T_r	Room temperature feedback
ac_unit.RH_r	Room relative humidity feedback
ac_unit.T_out	Outdoor temperature input
ac_unit.mdot_ac	Airflow state integrator
ac_unit.T_sup	Supply temperature state integrator
room_model.mdot_ac	ac_unit.mdot_out
room_model.T_sup	ac_unit.T_sup_out
room_model.RH_sup	ac_unit.RH_sup
room_model.dir_cmd	ac_unit.dir_out
room_model.T_out	Outdoor temperature input
room_model.RH_out	Outdoor relative humidity input
room_model.T_r	Room temperature state integrator
room_model.T_w	Wall temperature state integrator
room_model.T_f	Furniture temperature state integrator
room_model.w_r	Room humidity ratio state integrator

Table 4: Dynamic States

State	Input	Output
Airflow	dmdot_ac	mdot_ac
Supply temperature	dT_sup	T_sup
Room temperature	dT_r	T_r
Wall temperature	dT_w	T_w
Furniture temperature	dT_f	T_f
Humidity ratio	dw_r	w_r
Energy	P_ac	E_ac

2 Psychrometric Equations

Humidity is modeled using humidity ratio. Relative humidity is calculated as an output.

$$p_{sat}(T) = 610.94 \exp\left(\frac{17.625T}{T + 243.04}\right) \quad (1)$$

$$p_v = \phi p_{sat}(T) \quad (2)$$

$$w = 0.62198 \frac{p_v}{P_{atm} - p_v} \quad (3)$$

$$p_v = P_{atm} \frac{w}{0.62198 + w} \quad (4)$$

$$RH = 100 \frac{p_v}{p_{sat}(T)} \quad (5)$$

3 Controller Model

The controller calculates the desired room temperature internally:

$$T_{des} = \lim [24 + 0.08(T_{out} - 30), 23, 27] \quad (6)$$

$$RH_{des} = 50 \quad (7)$$

$$e_T = T_r - T_{des} \quad (8)$$

$$e_{RH} = RH_r - RH_{des} \quad (9)$$

Fan speed command:

$$fan_{cmd} = \begin{cases} 0, & e_T \leq 0.3 \text{ and } RH_r \leq 60 \\ 1, & 0.3 < e_T < 1.0 \\ 2, & 1.0 \leq e_T < 2.0 \\ 3, & e_T \geq 2.0 \end{cases} \quad (10)$$

Supply temperature command:

$$T_{sup,cmd} = \lim [T_{des} - 3 \max(e_T, 0) - 0.05 \max(e_{RH}, 0), T_{sup,min}, 22] \quad (11)$$

$$T_{sup,min} = \begin{cases} 22, & fan_{cmd} = 0 \\ 17, & fan_{cmd} = 1 \\ 15, & fan_{cmd} = 2 \\ 12, & fan_{cmd} = 3 \end{cases} \quad (12)$$

Air direction command:

$$dir_{cmd} = \lim [0.45 + 0.18 \max(e_T, 0) + 0.01 \max(e_{RH}, 0), 0.3, 1.0] \quad (13)$$

3.1 Controller MATLAB Function

```
1  function [fan_cmd, dir_cmd, T_sup_cmd, T_des, RH_des, e_T, e_RH] = controller( ...
2  t, T_r, RH_r, T_out, RH_out)
3
4  persistent fan_last dir_last Tsup_last t_last
5
6  if isempty(fan_last)
7  fan_last = 0;
8  dir_last = 0.5;
9  Tsup_last = 22;
10 t_last = -1e9;
11 end
12
13 dt_ctrl = 60;
14
15 if (t - t_last) < dt_ctrl
16 fan_cmd = fan_last;
17 dir_cmd = dir_last;
18 T_sup_cmd = Tsup_last;
19 T_des = calc_T_des(T_out);
20 RH_des = 50;
21 e_T = T_r - T_des;
22 e_RH = RH_r - RH_des;
23 return;
24 end
25
26 t_last = t;
27
28 RH_r = lim(RH_r, 0, 100);
29 RH_out = lim(RH_out, 0, 100);
30
31 T_des = calc_T_des(T_out);
32 RH_des = 50;
33
34 e_T = T_r - T_des;
35 e_RH = RH_r - RH_des;
36
37 T_dead = 0.3;
38 RH_high = 60;
39 RH_very_high = 70;
40
41 if e_T <= T_dead && RH_r <= RH_high
42 fan_cmd = 0;
43 elseif e_T > T_dead && e_T < 1.0
44 fan_cmd = 1;
45 elseif e_T >= 1.0 && e_T < 2.0
46 fan_cmd = 2;
47 else
48 fan_cmd = 3;
49 end
50
51 if RH_r > RH_high && T_r > (T_des - 0.5)
52 fan_cmd = max(fan_cmd, 1);
53 end
54
55 if RH_r > RH_very_high && T_r > (T_des - 1.0)
56 fan_cmd = max(fan_cmd, 2);
57 end
58
59 if T_r < (T_des - 0.4)
60 fan_cmd = 0;
61 end
62
63 if fan_cmd > fan_last + 1
64 fan_cmd = fan_last + 1;
65 elseif fan_cmd < fan_last - 1
66 fan_cmd = fan_last - 1;
```

```

67     end
68
69     fan_cmd = round(lim(fan_cmd, 0, 3));
70
71     if fan_cmd == 0
72         T_sup_cmd = 22;
73     else
74         T_sup_cmd = T_des - 3.0 * max(e_T, 0) - 0.05 * max(e_RH, 0);
75     end
76
77     if fan_cmd == 1
78         T_sup_min_eff = 17;
79     elseif fan_cmd == 2
80         T_sup_min_eff = 15;
81     elseif fan_cmd == 3
82         T_sup_min_eff = 12;
83     else
84         T_sup_min_eff = 22;
85     end
86
87     T_sup_cmd = lim(T_sup_cmd, T_sup_min_eff, 22);
88
89     if RH_r > RH_high && fan_cmd > 0
90         T_sup_cmd = min(T_sup_cmd, 17);
91     end
92
93     if RH_r > RH_very_high && fan_cmd > 0
94         T_sup_cmd = min(T_sup_cmd, 15);
95     end
96
97     if RH_out > 75 && e_T < 0.8 && RH_r < RH_high
98         fan_cmd = min(fan_cmd, 1);
99     end
100
101     dir_cmd = 0.45 + 0.18 * max(e_T, 0) + 0.01 * max(e_RH, 0);
102
103     if fan_cmd == 0
104         dir_cmd = 0.5;
105     end
106
107     dir_cmd = lim(dir_cmd, 0.3, 1.0);
108
109     fan_last = fan_cmd;
110     dir_last = dir_cmd;
111     Tsup_last = T_sup_cmd;
112
113     end
114
115     function T_des = calc_T_des(T_out)
116         T_des = 24 + 0.08 * (T_out - 30);
117         T_des = lim(T_des, 23, 27);
118     end
119
120     function y = lim(x, xmin, xmax)
121         y = min(max(x, xmin), xmax);
122     end

```

4 Air Conditioner Model

Fan airflow:

$$\dot{V}_{tar} = \begin{cases} 0.00, & fan_{cmd} = 0 \\ 0.06, & fan_{cmd} = 1 \\ 0.11, & fan_{cmd} = 2 \\ 0.16, & fan_{cmd} = 3 \end{cases} \quad (14)$$

$$\dot{m}_{tar} = \rho_a \dot{V}_{tar} \quad (15)$$

$$\frac{d\dot{m}_{ac}}{dt} = \frac{\dot{m}_{tar} - \dot{m}_{ac}}{\tau_{fan}} \quad (16)$$

Supply temperature dynamics:

$$\frac{dT_{sup}}{dt} = \frac{T_{sup,cap} - T_{sup}}{\tau_{coil}} \quad (17)$$

Cooling capacity:

$$\dot{Q}_{sens} = \dot{m}_{ac} c_{p,a} \max(T_r - T_{sup,tar}, 0) \quad (18)$$

$$\dot{Q}_{sens,lim} = \min(\dot{Q}_{sens}, \dot{Q}_{cool,max}) \quad (19)$$

Dehumidification:

$$T_{coil} = T_{sup} - \Delta T_{coil} \quad (20)$$

$$w_{sup} = B_{coil} w_r + (1 - B_{coil}) \min(w_r, w_{coil,sat}) \quad (21)$$

Electrical power:

$$h(T, w) = 1.006T + w(2501 + 1.86T) \quad (22)$$

$$P_{ac} = \frac{\dot{Q}_{tot}}{COP} + P_{fan} + P_{standby} \quad (23)$$

4.1 Air Conditioner MATLAB Function

```

1  function [dmdot_ac, dT_sup, mdot_out, T_sup_out, RH_sup, P_ac, dir_out] = ac_unit(
2      ...
3      fan_cmd, dir_cmd, T_sup_cmd, T_r, RH_r, T_out, mdot_ac, T_sup)
4
5      P_atm = 101325;
6      rho_a = 1.20;
7      cp_a = 1006;
8
9      Q_cool_max = 3500;
10     COP_nom = 3.2;
11     P_fan_max = 65;
12     P_stby = 5;
13
14     T_sup_min = 12;
15     T_sup_max = 24;
16     tau_fan = 8;
17     tau_coil = 60;
18     B_coil = 0.20;
19     dT_coil = 3.0;
20
21     fan_cmd = round(lim(fan_cmd, 0, 3));
22     dir_cmd = lim(dir_cmd, 0, 1);
23     RH_r = lim(RH_r, 0, 100);
24     T_sup_cmd = lim(T_sup_cmd, T_sup_min, T_sup_max);

```

```

24     mdot_ac = max(mdot_ac, 0);
25
26     if fan_cmd == 0
27         Vdot_tar = 0.00;
28     elseif fan_cmd == 1
29         Vdot_tar = 0.06;
30     elseif fan_cmd == 2
31         Vdot_tar = 0.11;
32     else
33         Vdot_tar = 0.16;
34     end
35
36     mdot_tar = rho_a * Vdot_tar;
37     mdot_max = rho_a * 0.16;
38
39     dmdot_ac = (mdot_tar - mdot_ac) / tau_fan;
40
41     if fan_cmd == 0
42         T_sup_tar = T_r;
43     else
44         T_sup_tar = T_sup_cmd;
45     end
46
47     T_sup_tar = lim(T_sup_tar, T_sup_min, T_r);
48
49     Q_sens_dem = mdot_ac * cp_a * max(T_r - T_sup_tar, 0);
50     Q_sens_lim = min(Q_sens_dem, Q_cool_max);
51
52     if mdot_ac > 1e-4
53         T_sup_cap = T_r - Q_sens_lim / (mdot_ac * cp_a);
54     else
55         T_sup_cap = T_r;
56     end
57
58     T_sup_cap = lim(T_sup_cap, T_sup_min, T_r);
59     dT_sup = (T_sup_cap - T_sup) / tau_coil;
60
61     mdot_out = lim(mdot_ac, 0, mdot_max);
62     T_sup_out = lim(T_sup, T_sup_min, T_r);
63
64     w_r = RH_to_w(T_r, RH_r, P_atm);
65     T_coil = T_sup_out - dT_coil;
66     w_coil_sat = RH_to_w(T_coil, 100, P_atm);
67
68     if fan_cmd == 0 || mdot_out < 1e-4
69         w_sup = w_r;
70     else
71         w_sup = B_coil * w_r + (1 - B_coil) * min(w_r, w_coil_sat);
72     end
73
74     RH_sup = lim(w_to_RH(T_sup_out, w_sup, P_atm), 0, 100);
75
76     h_ret = h_air(T_r, w_r);
77     h_sup = h_air(T_sup_out, w_sup);
78
79     Q_tot = mdot_out * max(h_ret - h_sup, 0) * 1000;
80     Q_tot = min(Q_tot, Q_cool_max);
81
82     COP = COP_nom - 0.04 * (T_out - 35);
83     COP = lim(COP, 1.5, 5.0);
84
85     if fan_cmd == 0 || Q_tot < 1
86         P_comp = 0;
87     else
88         P_comp = Q_tot / COP;
89     end
90
91     fan_ratio = mdot_out / mdot_max;
92     P_fan = P_fan_max * fan_ratio^3;
93
94     if fan_cmd == 0
95         P_ac = P_stby;
96     else
97         P_ac = P_comp + P_fan + P_stby;
98     end

```

```

99
100     dir_out = dir_cmd;
101
102     end
103
104     function pws = p_sat(T)
105     pws = 610.94 * exp((17.625 * T) / (T + 243.04));
106     end
107
108     function w = RH_to_w(T, RH, P_atm)
109     phi = lim(RH / 100, 0, 1);
110     pws = p_sat(T);
111     pv = min(phi * pws, 0.99 * P_atm);
112     w = 0.62198 * pv / (P_atm - pv);
113     w = max(w, 0.0001);
114     end
115
116     function RH = w_to_RH(T, w, P_atm)
117     w = max(w, 0.0001);
118     pws = p_sat(T);
119     pv = P_atm * w / (0.62198 + w);
120     RH = 100 * pv / pws;
121     end
122
123     function h = h_air(T, w)
124     h = 1.006 * T + w * (2501 + 1.86 * T);
125     end
126
127     function y = lim(x, xmin, xmax)
128     y = min(max(x, xmin), xmax);
129     end

```

5 Room Thermal and Humidity Model

Room states:

$$\mathbf{x} = [T_r \quad T_w \quad T_f \quad w_r]^T \quad (24)$$

Room air capacity:

$$V_r = LWH \quad (25)$$

$$C_r = \rho_a V_r c_{p,a} \quad (26)$$

Air direction effectiveness:

$$\eta_{dir} = 0.6 + 0.4 dir_{cmd} \quad (27)$$

Room thermal dynamics:

$$\frac{dT_r}{dt} = \frac{\dot{Q}_{ac} + \dot{Q}_{wr} + \dot{Q}_{fr} + \dot{Q}_{inf} + \dot{Q}_{int} + \dot{Q}_{sol,r}}{C_r} \quad (28)$$

$$\frac{dT_w}{dt} = \frac{\dot{Q}_{ow} - \dot{Q}_{wr} + \dot{Q}_{sol,w}}{C_w} \quad (29)$$

$$\frac{dT_f}{dt} = \frac{-\dot{Q}_{fr}}{C_f} \quad (30)$$

Room humidity dynamics:

$$\frac{dw_r}{dt} = \frac{\eta_{dir}\dot{m}_{ac}(w_{sup} - w_r) + \dot{m}_{inf}(w_{out} - w_r) + G_{moist}}{m_{da}} \quad (31)$$

5.1 Room MATLAB Function

```

1  function [dT_r, dT_w, dT_f, dw_r, RH_r] = room_model( ...
2  mdot_ac, T_sup, RH_sup, dir_cmd, ...
3  T_out, RH_out, T_r, T_w, T_f, w_r)
4
5  P_atm = 101325;
6  rho_a = 1.20;
7  cp_a = 1006;
8
9  L = 5.0;
10 W = 4.0;
11 H = 3.0;
12
13 V_r = L * W * H;
14 A_w = 2 * H * (L + W) + L * W;
15
16 x_w = 0.15;
17 rho_w = 1800;
18 cp_w = 840;
19 f_w = 0.35;
20
21 C_w = A_w * x_w * rho_w * cp_w * f_w;
22
23 h_ow = 8.0;
24 h_wr = 3.0;
25
26 R_ow = 1 / (h_ow * A_w);
27 R_wr = 1 / (h_wr * A_w);
28
29 m_f = 200;
30 cp_f = 1400;
31 C_f = m_f * cp_f;
32 R_fr = 0.08;
33
34 C_r = rho_a * V_r * cp_a;
35
36 ACH = 0.60;
37 V_inf = ACH * V_r / 3600;
38 mdot_inf = rho_a * V_inf;
39
40 Q_people = 100;
41 Q_light = 40;
42 Q_device = 80;
43 Q_int = Q_people + Q_light + Q_device;
44
45 G_moist_gph = 50;
46 G_moist = G_moist_gph / (1000 * 3600);
47
48 Q_sol_r = 0;
49 Q_sol_w = 0;
50
51 dir_cmd = lim(dir_cmd, 0, 1);
52 RH_sup = lim(RH_sup, 0, 100);
53 RH_out = lim(RH_out, 0, 100);
54 mdot_ac = max(mdot_ac, 0);
55 w_r = max(w_r, 0.0001);
56
57 eta_dir = 0.6 + 0.4 * dir_cmd;
58
59 w_out = RH_to_w(T_out, RH_out, P_atm);
60 w_sup = RH_to_w(T_sup, RH_sup, P_atm);
61
62 Q_ac = eta_dir * mdot_ac * cp_a * (T_sup - T_r);
63 Q_inf = mdot_inf * cp_a * (T_out - T_r);
64
65 Q_ow = (T_out - T_w) / R_ow;

```

```

66     Q_wr = (T_w - T_r) / R_wr;
67     Q_fr = (T_f - T_r) / R_fr;
68
69     dT_r = (Q_ac + Q_wr + Q_fr + Q_inf + Q_int + Q_sol_r) / C_r;
70     dT_w = (Q_ow - Q_wr + Q_sol_w) / C_w;
71     dT_f = (-Q_fr) / C_f;
72
73     m_da = rho_a * V_r;
74
75     dw_r = (eta_dir * mdot_ac * (w_sup - w_r) ...
76 + mdot_inf * (w_out - w_r) ...
77 + G_moist) / m_da;
78
79     RH_r = lim(w_to_RH(T_r, w_r, P_atm), 0, 100);
80
81     end
82
83     function pws = p_sat(T)
84     pws = 610.94 * exp((17.625 * T) / (T + 243.04));
85     end
86
87     function w = RH_to_w(T, RH, P_atm)
88     phi = lim(RH / 100, 0, 1);
89     pws = p_sat(T);
90     pv = min(phi * pws, 0.99 * P_atm);
91     w = 0.62198 * pv / (P_atm - pv);
92     w = max(w, 0.0001);
93     end
94
95     function RH = w_to_RH(T, w, P_atm)
96     w = max(w, 0.0001);
97     pws = p_sat(T);
98     pv = P_atm * w / (0.62198 + w);
99     RH = 100 * pv / pws;
100    end
101
102    function y = lim(x, xmin, xmax)
103    y = min(max(x, xmin), xmax);
104    end

```

6 Simulation Settings

Table 5: One-Day Simulation Settings

Setting	Value
Start time	0 s
Stop time	86400 s
Solver type	Fixed-step
Solver	ode4 Runge-Kutta
Fixed-step size	10 s
Controller update interval	60 s

7 Outdoor Input Data

```

1     t_day = (0:3600:86400)';
2
3     T_out_values = [ ...
4     24; 23.5; 23; 22.5; 22; 22.5; 24; 26; 29; 32; ...
5     34; 36; 37; 38; 37; 36; 34; 32; 30; 28; ...
6     27; 26; 25; 24.5; 24];
7
8     RH_out_values = [ ...
9     75; 78; 80; 82; 84; 83; 80; 74; 68; 60; ...
10    55; 48; 44; 42; 43; 46; 50; 56; 62; 67; ...

```

```

11     70; 72; 74; 75; 75];
12
13     T_out_input = timeseries(T_out_values, t_day);
14     RH_out_input = timeseries(RH_out_values, t_day);

```

8 Initial Conditions

Table 6: Initial Conditions

State	Initial condition	Unit
$\dot{m}_{ac}$	0	kg/s
T_{sup}	Initial room temperature	°C
T_r	Initial room temperature	°C
T_w	Initial room temperature	°C
T_f	Initial room temperature	°C
w_r	0.010	kg/kg
E_{ac}	0	J

9 Calibration Parameters

Table 7: Main Calibration Parameters

Parameter	Effect
Air changes per hour	Controls outdoor air leakage.
Wall active fraction	Controls wall thermal delay.
Furniture mass	Controls slow temperature response.
Cooling capacity	Limits maximum cooling effect.
Fan airflow values	Define airflow for each fan speed.
Coil bypass factor	Controls dehumidification strength.
Internal gains	Represent people, lights, devices, and moisture.